LEARNING TO REMEMBER FROM FAILURE: KNOWLEDGE, EXPERIENCE, AND WORKING MEMORY

Anonymous authors
Manuscript in preparation

ABSTRACT

Conversational memory can fail because evidence was discarded, relevant records were not retrieved, or retrieved evidence was used incorrectly. Repeating the same memory pipeline does not distinguish these failures. We propose a framework with two persistent banks and one transient working state: a knowledge bank stores source-grounded facts, an experience bank stores reusable repair procedures, and working memory tracks the evidence still needed for the current question. Experience controls how knowledge is encoded, retrieved, and used; only the knowledge bank can supply factual support for an answer. Failed development trajectories produce candidate procedures, which are retained only after controlled replay and validation. The model weights remain fixed. We specify three focused experiments on LoCoMo and LongMemEval: answer quality and cost, the contribution of the three memory roles, and transfer as experience accumulates. This manuscript presents the method and experimental protocol; benchmark results have not yet been measured.

1 INTRODUCTION

A memory-augmented agent may answer a question about a user's current residence with an old address. Three different failures can produce that answer: the move was omitted during memory construction, the new address was stored but not retrieved, or both addresses were retrieved without comparing their dates. These failures call for different interventions. An extra search cannot recover information that was never retained, while a better writer cannot by itself resolve a reader's temporal mistake.

Existing systems make complementary advances in memory extraction, organization, and efficient retrieval (Chhikara et al., 2025; Xu et al., 2025; Liu et al., 2026; Fang et al., 2025). Experience-based agents reuse feedback and prior attempts (Shinn et al., 2023; Zhang et al., 2026; Cao et al., 2025). Yet a conversational memory system still needs an explicit connection between *what it remembers* and *what it has learned about remembering*. We study that connection through three roles: persistent knowledge, persistent experience, and transient working state.

The proposed mechanism is *failure-directed memory repair*. A trace records which facts were available, what was retrieved, and which evidence supported the answer. A learner tests a small repair to the writer, retriever, or reader and distills a successful intervention into a reusable conditional procedure. Future questions retrieve these procedures as guidance, while their factual answers remain grounded in the current conversation's knowledge bank. Working memory connects the two banks by exposing missing evidence and failed attempts.

The memory taxonomy itself is established (LangChain, 2026). Recuris already uses working state to select experience and localizes failures for memory-control updates (Yu et al., 2026). Our proposed distinction is the repair of a separate *factual* bank and its evidence-access pipeline for conversational QA. This distinction is a hypothesis to test, rather than a claim of demonstrated superiority. The contribution of this draft is a concrete architecture, a bounded repair procedure, and an evaluation that separates factual recall from cross-question procedural transfer.

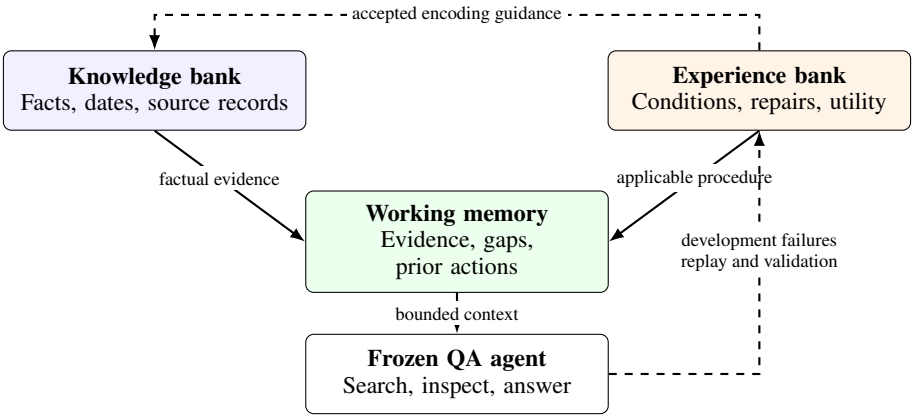


Figure 1: Two persistent banks and one transient state. Solid arrows are used while answering; dashed arrows describe development-time improvement and its effect on future memory construction. Source records are part of the knowledge bank, not an additional learned memory.

2 Three Memory Roles

Let H_i be the timestamped dialogue history associated with question q_i. The agent uses a frozen language model π_θ and memory

$$M_{i,t} = (\mathcal{K}_i, \mathcal{E}_n, w_{i,t}), \tag{1}$$

where $\mathcal{K}_i$ is scoped to the history, $\mathcal{E}_n$ contains experience accepted after development stage n, and $w_{i,t}$ exists only during the current question. Histories belonging to the same conversation may reuse their knowledge bank; unrelated users or evaluation folds cannot share facts. Figure 1 shows the two update timescales.

2.1 Knowledge: Preserve Claims and Their Sources

A knowledge record contains a claim, its participants, temporal scope, source turn identifiers, and a version. The same bank retains the original source records for bounded inspection or re-encoding. A writer processes history in chronological batches without access to future questions or gold answers. It preserves dated changes instead of replacing every old claim with the latest one. A correction links the revised claim to the affected version; incompatible claims remain explicit until their scopes can be resolved.

The default index combines dense and lexical retrieval. Accepted encoding procedures may change the writer's extraction instructions, such as preserving the time of a state change or retaining both participants in a relation. These procedures are selected from properties of the incoming batch, not from an unseen evaluation question. At query time, a bounded source inspection may recover an omitted distinction. Such access is logged and charged to the retrieval and context budgets.

2.2 Experience: Store Conditional Repair Procedures

An experience entry has the form

$$e = (c, z, p, u), \qquad z \in \{\text{encode}, \text{retrieve}, \text{answer}\}, \tag{2}$$

where c specifies its applicability, z names the operation to change, p is a short procedure, and u records validation support and subsequent successes or failures. For example, a procedure may require searching adjacent dated sessions when a question asks about a change of state. It must not store a benchmark answer as the procedure. Names, answer strings, and conversation identifiers are removed from reusable guidance; development source identifiers remain in an audit record that is not exposed as answer context.

Applicability is filtered by operation and observable problem features before semantic ranking. Retrieval returns at most two procedures. A validated procedure has priority over an untested candidate, and a contradicted or repeatedly unhelpful entry is retired. This bank transfers problem-solving

experience across conversations, while the knowledge bank supplies the facts for the current conversation.

2.3 Working State: Track Evidence and Unresolved Needs

Working memory is a compact record of the question, evidence identifiers, unresolved entities or dates, conflicts, attempted operations, and remaining budget. It is initialized for each question and discarded afterward. The agent does not persist a free-form reasoning transcript as factual memory.

At step t, the controller retrieves applicable experience and updates the working state from the actual retrieval or inspection result:

$$\begin{aligned} P_t &= \mathrm{RetrieveExperience}(\mathcal{E}_n, q_i, w_{i,t}), \\ (R_t, a_t) &= \mathrm{Act}(\pi_\theta, q_i, w_{i,t}, P_t, \mathcal{K}_i), \\ w_{i,t+1} &= \mathrm{Update}(w_{i,t}, R_t, a_t). \end{aligned} \tag{3}$$

Available actions are knowledge search, source inspection, comparison of retrieved evidence, and answer or abstain. A missing fact is marked resolved only when a returned source supports it. The agent stops when the evidence is sufficient or the action budget is exhausted. Experience may guide the search or interpretation, but cannot itself serve as factual evidence.

3 Learning from Failure

The experience bank starts empty. Development runs log the knowledge version, retrieved records, working states, actions, cited sources, answer, and feedback. Gold answers and evidence annotations may supervise this development process; they are unavailable during held-out inference. A failure opens a small, predefined repair space:

Repair target	Diagnostic evidence	Candidate intervention
Encoding	Relevant source exists, but its claim or scope is missing from the bank	Re-encode a bounded source region with revised extraction guidance
Retrieval	Supporting records exist but were absent from the answer context	Change query decomposition, entity expansion, or temporal filtering
Answering	Supporting evidence reached the reader but was misused	Require evidence comparison, date resolution, or calibrated abstention

The learner proposes one local procedure for each implicated target and replays the failed development examples with the model, sources, and resource limits held fixed. A probe changes one operation at a time. Its outcome indicates a useful intervention, not a uniquely identified causal explanation; ambiguous failures may remain unresolved. An encoding repair must produce source-backed records, and an answering repair cannot inject the gold answer into the prompt.

Let F be the failed development batch and V a separate validation set. A candidate procedure p is admitted only when it improves mean QA score on F, does not reduce mean QA score on V, and respects the resource limits:

$$\mathrm{Accept}(p) = \mathbf{1}\{\Delta_F(p) > 0 \ \wedge \ \Delta_V(p) \geq 0 \ \wedge \ C(p) \leq B\}. \tag{4}$$

Replays use matched seeds and starting knowledge snapshots. A failed candidate does not alter deployed experience. The writer is versioned with the accepted encoding procedures so that future banks can be rebuilt from their histories. Within a frozen evaluation run, source inspection changes only the current working context; answering one question cannot improve the bank for the next.

The loop is bounded to three development rounds and a bank of at most 64 procedures. It retains generalized guidance rather than an ever-growing set of answer examples. The learner receives validation acceptance decisions, while test histories, questions, labels, and scores never enter procedure generation or selection. This is continual adaptation through external memory; no model parameters are optimized.

4 Experimental Protocol

Empirical status. The experiments below are specified but not executed. No accuracy, cost reduction, significance claim, or synthetic performance curve is reported as an observed result.

Benchmarks and separation. We will evaluate the released LoCoMo conversational QA data (Maharana et al., 2024) and the cleaned LongMemEval-S data (Wu et al., 2024). LoCoMo tests single-hop, multi-hop, temporal, open-domain, and adversarial questions; LongMemEval tests long histories, updates, temporal reasoning, and cross-session recall. The primary protocol uses five outer folds. Each LoCoMo fold assigns six whole conversations to experience development, two to validation, and two to test; each conversation is tested once. LongMemEval uses approximately 60/20/20 percent development/validation/test per fold, grouping examples with shared user histories or answer-bearing sessions before assignment. Every released example is evaluated only in its held-out fold. Exact group sizes and manifests must be published. These are cross-fitted research scores, not the official fixed-bank leaderboard setting.

Each test history is ingested from scratch using the frozen experience learned outside its fold. Neither benchmark's test feedback updates either bank. Chronological batches within the development partition supply the experience stream; this ordering is an experimental schedule, not a claim that unrelated benchmark histories form one natural user timeline.

Shared configuration and comparison. The planned primary reader and learner are Qwen3-8B, with Qwen3-Embedding-4B for dense retrieval. Temperature is zero and paired trials use three seeds. The primary per-step memory budget is 4,096 reader tokens, allocated as up to 3,072 for evidence, 512 for experience, and 512 for working state. Unused space is available to evidence. A question permits at most three retrieval or inspection actions; the same total memory budget and action ceiling apply to controlled baselines. We will report the 2,048/8,192-token budgets only as a sensitivity curve, and cap experience development at three rounds.

Comparisons include raw-history hybrid RAG, Mem0, A-MEM, SimpleMem, LightMem, and a LangMem store manager. Every method uses the same reader and embedding models. We will pin source commits and report any adapters or deviations from native execution; a modified implementation will not be labeled a paper reproduction. Full-context answering is a separate reference with its larger input budget explicitly reported. An additional controlled EM–WM baseline uses experience and working state with fixed knowledge encoding, inspired by Recuris; it is an adaptation for QA, not an official Recuris result.

4.1 Answer Quality and Cost

The main comparison will report LoCoMo token F1 over the four answerable categories, category-level scores, and adversarial abstention separately. LongMemEval uses the official question-type-aware judging procedure. A pinned GPT-4o judge is planned, with its exact snapshot, prompt, and raw decisions recorded; the same judge is used across systems. Evidence recall is reported from source turns for LoCoMo and supporting sessions for LongMemEval, with abstention cases excluded from evidence recall. These units are not pooled. An experience entry never counts as retrieved factual support.

One main results table will pair answer quality with retrieved tokens, median and tail latency, and total model calls. Construction, development replay, and query-time costs will be shown separately, alongside amortized cost per answered question. Paired uncertainty estimates will resample entire conversations or shared-history groups, rather than treating correlated questions as independent.

4.2 Contribution of the Three Memories

A single controlled ablation table will compare the full framework with: knowledge only; knowledge plus experience without an explicit working state; knowledge plus working state with no learned experience; and one pooled store containing the same knowledge and procedure records. All variants keep the same source archive, action ceiling, reader, and total token budget; removed memory allocations return to factual evidence. A fixed, length-matched set of human-written procedures tests whether learned experience helps beyond generic advice. We will also disable encoding repair while

keeping the rest of the loop intact to isolate improvement of factual storage from query-time control. Failure cases will be inspected through the logged availability, retrieval, and use of the annotated evidence.

4.3 Continual Transfer and Memory Growth

We will evaluate frozen experience snapshots after each development round on untouched test folds without selecting a snapshot by test score. A single curve will show answer quality against accumulated experience and total development cost; a companion retention measure will track previously solved validation questions. A cross-benchmark test will learn experience on LoCoMo and apply it to LongMemEval, and reverse the direction, rebuilding all target knowledge banks from their own histories without target feedback. This separates procedural transfer from memorizing a benchmark's answers. Source-side validation fixes all settings before the transfer run.

5 Related Work and Scope

Fact-oriented systems extract, connect, and consolidate information for later recall, including Mem0, A-MEM, SimpleMem, and LightMem (Chhikara et al., 2025; Xu et al., 2025; Liu et al., 2026; Fang et al., 2025). Our knowledge bank adopts this setting; the proposed addition is learning conditional interventions from its failures. Reflexion, AdaMEM, and ReMe reuse feedback or execution experience (Shinn et al., 2023; Zhang et al., 2026; Cao et al., 2025). LangMem exposes semantic, episodic, and procedural memory as configurable primitives (LangChain, 2026).

Recuris is the closest control-level comparison: it couples experience with verified working state and updates skills, state specifications, invocation policies, and checkers from failed traces (Yu et al., 2026). We do not claim these general principles as new. Our specific target is factual omission and evidence access in a separately scoped conversational knowledge bank. The encoding-repair ablation and the fixed-encoding EM–WM control are necessary to determine whether that distinction has measurable value.

6 Limitations and Conclusion

The design depends on the fidelity of source attribution and development feedback. Replay can identify a helpful repair without proving the true cause of failure. Repeated validation may overfit a small development set, and LoCoMo has few independent conversations. Keeping source records increases storage cost; learned procedures may also transfer poorly across domains. These limitations must be evaluated rather than assumed away.

The framework separates factual support, reusable experience, and current evidence state. Its central hypothesis is that experience should change how knowledge is retained and accessed, while remaining distinct from the facts used to answer. LoCoMo and LongMemEval provide a focused test of that hypothesis. Until those experiments are completed, effectiveness remains an open empirical question.

AI Use Statement

Generative AI assisted the development of the proposed architecture, literature comparison, experimental protocol, prose, and LaTeX for this working draft. No framework implementation, benchmark runs, or empirical results were generated for this manuscript. Human authors must review the method, references, and final disclosure before submission.

REFERENCES

Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember Me, Refine Me: A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. arXiv:2512.10696, 2025. URL https://arxiv.org/abs/2512.10696.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025. URL https://arxiv.org/abs/2504.19413.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and Efficient Memory-Augmented Generation. arXiv:2510.18866, 2025. URL https://arxiv.org/abs/2510.18866.

LangChain. LangMem: Core Concepts, 2026. URL https://langchain-ai.github.io/langmem/concepts/conceptual_guide/. Online documentation, accessed September 3, 2026.

Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient Lifelong Memory for LLM Agents. arXiv:2601.02553, 2026. URL https://arxiv.org/abs/2601.02553.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753, 2024. URL https://arxiv.org/abs/2402.17753.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023. URL https://arxiv.org/abs/2303.11366.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813, 2024. URL https://arxiv.org/abs/2410.10813.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110, 2025. URL https://arxiv.org/abs/2502.12110.

Zhaochen Yu, Yingcheng Wu, Zhenfei Yin, Kaiyuan Chen, Zhe Zhao, Mengdi Wang, Shuicheng Yan, and Ling Yang. Recursive Experiential-Working Memory Evolution for Long-Horizon Agent Harnesses. arXiv:2608.24876, 2026. URL https://arxiv.org/abs/2608.24876.

Yunxiang Zhang, Yiheng Li, Ali Payani, and Lu Wang. AdaMEM: Test-Time Adaptive Memory for Language Agents. arXiv:2606.05684, 2026. URL https://arxiv.org/abs/2606.05684.